

■ 강의명: CSCI89

■ 주차: Lecture 5

■ 교수명: UIUC Student

■ 목적: 이 문서는 CSCI89 Lecture 5 강의 내용을 바탕으로 텍스트 처리의 핵심 개념과 기술을 다룹니다. 주요 목표는 Bag-of-Words의 한계를 극복하고, 단어의 의미론적 관계를 포착하는 방법을 이해하는 것입니다. 가중치 공유, N-그램 토큰화, TF-IDF, 단어 임베딩(Word2Vec, GloVe) 등의 개념을 상세히 설명하고, 실제 코드 예시를 통해 구현 방법을 제시합니다. 시험 대비를 위해 비전공자도 쉽게 이해할 수 있도록 직관적인 설명과 풍부한 예시를 포함합니다.

Contents

1	용어 정리	2
2	강의 및 과제 일정 안내	3
3	핵심 개념 및 원리	4
3.1	퀴즈 4 해설	4
3.1.1	1. 가중치 공유 (Weight Sharing)	4
3.1.2	2. N-그램 토큰화 (N-gram Tokenization)	5
3.1.3	3. N-그램 개수 계산	5
3.1.4	4. 합성곱 신경망 (CNN) 패딩과 스트라이드	6
4	5. TF-IDF (Term Frequency-Inverse Document Frequency)	8
4.0.1	5.1. Term Frequency (TF)	8
4.0.2	5.2. Inverse Document Frequency (IDF)	8
4.0.3	5.3. TF-IDF 계산 및 예시	9
4.0.4	5.4. TF-IDF 변형 및 정규화	10
5	6. 단어 임베딩 (Word Embedding)	12
5.0.1	6.1. 원-핫 인코딩 (One-Hot Encoding)의 한계	12
5.0.2	6.2. 임베딩 레이어 (Embedding Layer)	12
5.0.3	6.3. 임베딩 차원 결정	13
5.0.4	6.4. 임베딩의 의미론적 특성	13
5.0.5	6.5. 단어 임베딩의 활용	13
5.0.6	6.6. 단어 임베딩의 한계	13
5.0.7	6.7. 인기 있는 임베딩 모델	14
6	절차/방법: 텍스트 처리 실습	16
6.1	1. TF-IDF 벡터화 (scikit-learn TfidfVectorizer)	16
6.1.1	1.1. TfidfVectorizer 주요 파라미터	16
6.1.2	1.2. 불용어 (Stop Words) 제거	17
6.1.3	1.3. N-그램 적용	18
6.2	2. NLTK를 이용한 텍스트 전처리 (스테밍)	19
7	3. Word2Vec 임베딩 (gensim Word2Vec)	21

7.0.1	3.1. Word2Vec 주요 파라미터	21
7.0.2	3.2. 단어 유사성 계산	22
7.0.3	3.3. 단어 유추 (Analogy) 해결	23
8	실습/코드/오류와 해결	24
8.1	1. TF-IDF 벡터 정규화 논의	24
8.2	2. ChatGPT 언어별 성능 논의	24
9	체크리스트	26
10	FAQ (자주 묻는 질문)	27
11	빠르게 훑어보기 (1페이지 요약)	28

1 용어 정리

용어	쉬운 설명	원어	비고
가중치 공유	신경망의 일부 가중치를 여러 위치에서 재사용하여 모델 복잡도 감소	Weight Sharing	RNN, CNN에서 주로 사용
N-그램	텍스트에서 N개의 연속된 단어 묶음	N-gram	단어 순서 정보를 일부 보존
Bag-of-Words	문서 내 단어의 출현 빈도만으로 문서를 표현하는 방식	Bag-of-Words (BoW)	단어 순서 정보 손실, 희소 표현
TF-IDF	단어의 중요도를 측정하는 통계적 방법	Term Frequency-Inverse Document Frequency	TF(빈도)와 IDF(희귀성)의 곱
Term Frequency (TF)	특정 문서 내에서 단어가 나타나는 빈도	Term Frequency	문서 길이에 대한 상대적 빈도
Inverse Document Frequency (IDF)	전체 문서 집합에서 단어가 얼마나 희귀한지 나타내는 값	Inverse Document Frequency	희귀할수록 높은 값
원-핫 인코딩	단어를 고유한 위치에 1을, 나머지는 0으로 표현하는 방식	One-Hot Encoding	희소하고 의미 관계를 반영 못함
단어 임베딩	단어를 저차원의 밀집 벡터로 표현하여 의미 관계를 포착	Word Embedding	Word2Vec, GloVe 등
임베딩 레이어	신경망에서 원-핫 벡터를 임베딩 벡터로 변환하는 층	Embedding Layer	학습 가능한 가중치 행렬
Word2Vec	단어 임베딩을 학습하는 모델 (Google)	Word2Vec	CBOW, Skip-gram 방식
GloVe	단어 임베딩을 학습하는 모델 (Stanford)	Global Vectors for Word Representation	동시 출현 행렬 기반
CBOW	주변 단어를 이용해 중심 단어를 예측하는 Word2Vec 방식	Continuous Bag-of-Words	
Skip-gram	중심 단어를 이용해 주변 단어를 예측하는 Word2Vec 방식	Skip-gram	
코사인 유사성	두 벡터 간의 각도를 이용해 유사도를 측정하는 방법	Cosine Similarity	임베딩 벡터 간 유사도 측정에 활용
패딩 (Padding)	합성곱 연산 시 이미지 가장자리에 0을 추가하는 기법	Padding	출력 차원 유지 또는 경계 정보 보존
스트라이드 (Stride)	합성곱 필터가 이동하는 간격	Stride	출력 차원 조절
오토인코더	입력 데이터를 압축 후 다시 복원하는 신경망	Autoencoder	데이터 압축, 노이즈 제거 등에 활용
스테밍 (Stemming)	단어의 어간을 추출하여 단어를 정규화하는 기법	Stemming	'cats' → 'cat'
표제어 추출	단어의 기본형(표제어)을 찾아 단어를 정규화하는 기법	Lemmatization	'better' → 'good'

2 강의 및 과제 일정 안내

과제 마감일 변경

다음 과제 마감일이 변경되었습니다. 원래 이번 주 일요일(This Sunday)이 아닌, **다다음 주 일요일(The following Sunday)**로 연기되었습니다. 이는 여러분이 과제를 작업할 수 있는 시간을 약 2주 정도 확보해 주기 위함입니다.

- **과제 게시:** 이미 과제는 게시되었지만, 마감일이 연기되었습니다.
- **변경 이유:** 강의에서 다루는 내용(예: TF-IDF)이 과제에 바로 적용되기 때문에, 강의와 과제 마감일 사이에 충분한 시간(최소 1주)을 두어 학생들이 편안하게 학습하고 과제를 수행할 수 있도록 하기 위함입니다.
- **전체 과제 일정 조정:** 이번 과제 마감일 연기로 인해 전체 과제 일정이 뒤로 밀리게 됩니다. 이는 Excel 문서에 업데이트된 스케줄로 공지되었습니다. 과제 개수 자체가 줄어드는 것이 아니라, 단순히 마감일이 순차적으로 뒤로 밀리는 것입니다.
- **누적 과제 없음:** 다음 주에 두 개의 과제를 제출해야 하는 등의 누적은 없습니다. 각 과제에 충분한 시간을 주기 위해 전체 일정이 조정된 것입니다.

3 핵심 개념 및 원리

3.1 퀴즈 4 해설

3.1.1 1. 가중치 공유 (Weight Sharing)

가중치 공유는 신경망 모델의 파라미터 수를 줄이고, 특정 패턴을 여러 위치에서 재사용할 수 있도록 하는 중요한 개념입니다. 이는 모델의 일반화 성능을 높이고 과적합을 방지하는 데 도움을 줍니다.

가중치 공유의 개념

가중치 공유(Weight Sharing)는 신경망의 여러 연결에서 동일한 가중치(W)를 사용하는 기법입니다. 이는 모델의 파라미터 수를 크게 줄여 계산 효율성을 높이고, 특정 특징(패턴)이 입력 데이터의 위치에 관계없이 중요하다고 가정할 때 유용합니다.

- **완전 연결 신경망 (Fully Connected Neural Network, FNN):**
 - 설명: 가장 기본적인 신경망으로, 이전 레이어의 모든 뉴런이 다음 레이어의 모든 뉴런과 연결됩니다. 각 연결에는 고유한 가중치(W)가 할당됩니다.
 - 가중치 공유 여부: 가중치 공유가 없습니다. 모든 W_{ij} 는 고유합니다.
 - 문제점: 입력 차원이 커지면 파라미터 수가 기하급수적으로 늘어나 학습이 어렵고 과적합 위험이 높습니다.
- **순환 신경망 (Recurrent Neural Network, RNN):**
 - 설명: 시퀀스 데이터(시간, 텍스트 등) 처리에 특화된 신경망입니다. 현재 시점의 출력이 다음 시점의 입력으로 다시 들어가는 '순환' 구조를 가집니다.
 - 가중치 공유 여부: 가중치 공유가 발생합니다. 서로 다른 시점의 입력에 대해 동일한 가중치 집합을 사용합니다.
 - 이유: 시퀀스 데이터에서 시간 독립적인 패턴(예: 문법 규칙)을 학습하기 위함입니다. 즉, 특정 패턴이 시퀀스의 어느 시점에서 나타나든 동일하게 인식되어야 한다고 가정합니다. 이는 차원 축소의 한 방법이기도 합니다.
 - 예시: "나는 학교에 간다"라는 문장에서 '나는' 다음에 '학교에'가 오는 패턴은 문장의 시작이든 중간이든 동일한 의미를 가질 수 있습니다. RNN은 이 패턴을 학습하는 가중치를 모든 시점에서 공유합니다.
- **합성곱 신경망 (Convolutional Neural Network, CNN):**
 - 설명: 이미지와 같은 그리드(grid) 형태의 데이터 처리에 특화된 신경망입니다. 필터(커널)를 입력 데이터에 슬라이딩하며 특징 맵(feature map)을 생성합니다.
 - 가중치 공유 여부: 가중치 공유가 발생합니다. 하나의 필터가 이미지의 여러 영역에 걸쳐 동일한 가중치 집합을 사용하여 특징을 추출합니다.
 - 이유: 이미지에서 특정 특징(예: 모서리, 특정 모양)이 이미지의 어느 위치에 나타나든 동일한 특징으로 인식되어야 한다고 가정합니다. 필터가 이미지를 '슬라이딩'하는 과정 자체가 가중치 공유를 의미합니다.
 - 예시: 고양이 이미지에서 '눈'을 인식하는 필터는 이미지의 왼쪽 상단에서든 오른쪽 하단에서든 동일한 가중치를 사용하여 '눈'이라는 특징을 찾아냅니다.

3.1.2 2. N-그램 토큰화 (N-gram Tokenization)

텍스트를 분석할 때 단어 하나하나를 보는 것을 넘어, 여러 단어의 연속된 묶음을 함께 고려하는 것이 N-그램 토큰화입니다. 이는 단어의 순서 정보를 일부 보존하여 문맥을 더 잘 이해하는 데 도움을 줍니다.

N-그램 토큰화의 개념

N-그램(N-gram)은 텍스트에서 N 개의 연속된 단어(또는 토큰)를 하나의 단위로 묶는 방법입니다. **2-그램(bi-gram)**은 두 개의 연속된 단어를 묶으며, 이는 단어의 순서 정보를 부분적으로 보존하여 문맥을 파악하는 데 유용합니다.

- **Bag-of-Words의 한계:** Bag-of-Words는 단어의 출현 빈도만 고려하고 순서는 무시합니다. "not good"과 "good not"이 동일하게 처리될 수 있습니다.
- **N-그램의 필요성:** N-그램은 이러한 순서 정보를 일부 보존하여 "not good"과 같은 부정적인 표현을 하나의 의미 단위로 인식할 수 있게 합니다.
- **오버랩핑 N-그램 (Overlapping N-grams):** N-그램을 추출할 때는 일반적으로 오버랩핑 방식을 사용합니다. 즉, 한 N-그램의 마지막 단어가 다음 N-그램의 첫 단어가 될 수 있습니다. 이는 모든 가능한 N-그램 조합을 포착하기 위함입니다.
- **예시 문장 분석:**

"Port assembly line one reduced car costs"

이 문장에서 2-그램을 추출하면 다음과 같습니다 (오버랩핑 방식):

- "Port assembly"
- "assembly line"
- "line one"
- "one reduced"
- "reduced car"
- "car costs"

강의에서 제시된 퀴즈의 정답은 이와 같이 모든 오버랩핑 2-그램을 포함하는 첫 번째 옵션이었습니다.

3.1.3 3. N-그램 개수 계산

주어진 텍스트에서 N-그램의 총 개수를 계산하는 것은 N-그램 모델을 구축할 때 중요한 단계입니다.

N-그램 개수 계산 공식

길이가 L 인 텍스트에서 N -그램의 개수는 일반적으로 $L - N + 1$ 입니다. 단, 모든 N -그램이 고유하다는 가정 하에 계산됩니다.

- **예시:** 5개의 연속된 단어로 구성된 텍스트 "A B C D E"
 - **1-그램 (단어):** A, B, C, D, E (총 5개)
 - **2-그램 (bi-gram):** AB, BC, CD, DE (총 $5 - 2 + 1 = 4$ 개)
 - **3-그램 (tri-gram):** ABC, BCD, CDE (총 $5 - 3 + 1 = 3$ 개)
- **경계 효과 (Boundary Effect):** 텍스트의 시작과 끝 부분에서는 N-그램을 만들 수 있는 단어의 수가 줄어들기 때문에, N-그램의 개수가 1-그램(단어)의 개수보다 적을 수 있습니다.
- **실제 어휘 수:** 이론적으로는 N-그램의 개수가 단어의 개수보다 적을 수 있지만, 실제 대규모 텍스트

(코퍼스)에서는 N-그램의 **고유한(unique)** 개수가 1-그램의 고유한 개수보다 훨씬 많습니다.

- **예시:** 'computer'라는 단어 뒤에는 'is fast', 'does not respond', 'does' 등 다양한 단어가 올 수 있습니다. 'computer'는 1-그램으로 하나지만, 'computer is', 'computer does' 등 2-그램은 훨씬 많아질 수 있습니다.
- **의미:** N-그램을 사용하면 어휘(vocabulary)의 크기가 매우 커지므로, 모델의 복잡도가 증가하고 학습에 더 많은 데이터와 계산 자원이 필요합니다.

3.1.4 4. 합성곱 신경망 (CNN) 패딩과 스트라이드

CNN에서 패딩(padding)과 스트라이드(stride)는 합성곱 연산의 출력 차원을 조절하는 중요한 하이퍼파라미터입니다.

패딩과 스트라이드의 역할

패딩(Padding)은 입력 데이터의 가장자리에 0을 추가하여 출력 특징 맵의 크기를 조절합니다. **스트라이드(Stride)**는 필터가 입력 데이터를 가로지르는 간격을 의미하며, 이 또한 출력 특징 맵의 크기에 영향을 줍니다.

- **padding='same'과 strides=1:**
 - **목표:** 입력 데이터의 공간적 차원(가로, 세로)을 출력 특징 맵에서 **동일하게 유지**하는 것입니다.
 - **방법:** padding='same'은 필터 크기에 맞춰 자동으로 0을 추가하여 입력과 출력의 차원을 같게 만듭니다. strides=1은 필터가 한 칸씩 이동하므로, 모든 픽셀을 한 번씩 처리합니다.
 - **활용:** 오토인코더(Autoencoder)와 같이 입력과 출력의 크기를 동일하게 유지해야 하는 대칭적인 신경망 구조에서 널리 사용됩니다.
- **padding='valid'와 strides > 1:**
 - **padding='valid':** 패딩을 전혀 추가하지 않습니다. 이 경우 출력 특징 맵의 크기는 입력보다 작아 집니다.
 - **strides > 1:** 필터가 여러 칸씩 건너뛰며 이동합니다. 이 경우 출력 특징 맵의 크기는 입력보다 크게 줄어듭니다.
 - **목표:** 주로 특징 맵의 차원을 줄여서 모델의 복잡도를 낮추고 계산량을 줄이는 데 사용됩니다.
- **Max Pooling (최대 풀링):**
 - **설명:** 합성곱 레이어 다음에 오는 다운샘플링(downsampling) 기법으로, 특정 영역(예: 2x2) 내에서 가장 큰 값을 추출하여 특징 맵의 크기를 줄입니다.
 - **기본 동작:** Max Pooling은 기본적으로 strides가 풀링 필터 크기와 동일하게 설정되어 영역들이 겹치지 않도록 합니다 (예: 2x2 풀링에 strides=2).
 - **비대칭성:** 입력 데이터의 크기가 풀링 필터 크기로 나누어 떨어지지 않을 경우, 남는 가장자리 픽셀들은 **버려집니다**. 이는 패딩이 없는 경우에도 발생하는 비대칭적인 동작입니다.
- **오토인코더 (Autoencoder)에서의 활용:**
 - 오토인코더는 입력 이미지를 압축(인코딩)했다가 다시 원본 이미지로 복원(디코딩)하는 신경망입니다.
 - 인코더 부분에서는 합성곱 레이어와 풀링을 통해 차원을 줄이고, 디코더 부분에서는 역합성곱(deconvolution) 또는 전치 합성곱(transposed convolution)을 통해 차원을 다시 늘립니다.
 - 이때, padding='same'과 strides=1을 사용하여 중간 합성곱 레이어들의 크기를 동일하게 유지하면, 원본 이미지 크기를 복원하기가 훨씬 용이해집니다. 이는 '샌드위치' 아키텍처라고도 불리며,

이미지 노이즈 제거(denoising autoencoder)와 같은 작업에 유용합니다.

4 5. TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF는 텍스트 마이닝에서 단어의 중요도를 측정하는 데 널리 사용되는 통계적 가중치 기법입니다. 단순히 단어의 출현 빈도만 보는 Bag-of-Words의 한계를 극복합니다.

TF-IDF의 개념 및 필요성

TF-IDF는 특정 단어가 문서 내에서 얼마나 자주 나타나는지(TF)와 전체 문서 집합에서 얼마나 희귀하게 나타나는지(IDF)를 결합하여 단어의 중요도를 측정합니다. 이는 '흔한 단어는 중요하지 않고, 특정 문서에서 자주 나오면서 다른 문서에서는 잘 나오지 않는 단어가 중요하다'는 직관을 반영합니다.

• Bag-of-Words의 한계:

- 단순히 단어의 출현 횟수(count)만 사용합니다.
- 'the', 'a', 'is'와 같은 불용어(stop words)나 'Boston'과 같이 특정 도메인에서 너무 흔한 단어들은 모든 문서에 자주 나타나지만, 문서의 내용을 구별하는 데는 거의 도움이 되지 않습니다. Bag-of-Words는 이러한 단어들에 높은 가중치를 부여하게 됩니다.

• TF-IDF의 목표:

- 문서 내에서 자주 나타나는 단어에 높은 가중치를 부여합니다 (TF).
- 전체 문서 집합에서 희귀하게 나타나는 단어에 높은 가중치를 부여합니다 (IDF).
- 결과적으로, 특정 문서에서 자주 나타나지만 다른 문서에서는 잘 나타나지 않는 단어에 가장 높은 가중치를 부여하여 해당 문서의 특징을 잘 나타내도록 합니다.

4.0.1 5.1. Term Frequency (TF)

TF는 특정 문서 내에서 단어가 얼마나 자주 나타나는지를 측정합니다. 문서의 길이에 따라 단어 빈도가 달라질 수 있으므로, 일반적으로 문서 길이로 정규화된 빈도를 사용합니다.

Term Frequency (TF) 정의

Term Frequency (TF)는 특정 단어(t)가 특정 문서(d) 내에서 나타나는 횟수를 해당 문서의 총 단어 수로 나눈 값입니다.

$$TF(t, d) = \frac{\text{단어 } t \text{의 문서 } d \text{ 내 출현 횟수}}{\text{문서 } d \text{의 총 단어 수}}$$

- **예시:** 문서 d 가 총 3개의 단어로 구성되어 있고, 'cat'이라는 단어가 2번 나타났다면, $TF('cat', d) = 2/3$ 입니다.
- **목표:** 문서의 길이가 길수록 특정 단어가 더 많이 나타날 가능성이 높으므로, 문서 길이로 나누어 상대적인 빈도를 측정합니다.

4.0.2 5.2. Inverse Document Frequency (IDF)

IDF는 특정 단어가 전체 문서 집합(코퍼스)에서 얼마나 희귀하게 나타나는지를 측정합니다. 희귀한 단어일수록 높은 IDF 값을 가집니다.

Inverse Document Frequency (IDF) 정의

Inverse Document Frequency (IDF)는 전체 문서 집합(N)에서 특정 단어(t)를 포함하는 문서의 수($DF(t)$)에 반비례하는 값입니다. 일반적으로 로그 스케일로 계산됩니다.

$$IDF(t) = \log \left(\frac{\text{전체 문서 수 } N}{\text{단어 } t \text{를 포함하는 문서 수 } DF(t)} \right)$$

- **로그 스케일링의 이유:**
 - 매우 희귀한 단어(예: 100만 개 문서 중 1개 문서에만 출현)에 과도하게 높은 IDF 값이 부여되는 것을 방지합니다. 로그 함수는 값이 커질수록 증가율이 둔화되므로, 희귀 단어의 가중치를 적절히 조절합니다.
 - 또한, IDF 값이 너무 커지면 TF-IDF 값 전체가 특정 희귀 단어에 의해 지배될 수 있습니다.
- **분모가 0이 되는 경우 방지:**
 - $DF(t)$ 는 단어 t 를 포함하는 문서의 수이므로, 우리가 사용하는 어휘(vocabulary)에 있는 단어라면 최소한 1개 이상의 문서에 나타나므로 0이 될 일은 없습니다.
 - 하지만 안전을 위해 $DF(t)$ 에 1을 더하는 변형($\log(N/(1 + DF(t)))$)을 사용하기도 합니다.
- **훈련 데이터셋 기반 계산:**
 - IDF는 단어 자체의 특성(코퍼스 내 희귀성)을 나타내므로, **훈련 데이터셋(train data set)**을 기반으로 한 번만 계산됩니다.
 - 이후 테스트 데이터셋(test data set)의 문서를 처리할 때도 훈련 데이터셋에서 계산된 IDF 값을 그대로 사용합니다.
 - **이유:** 테스트 데이터셋은 모델이 처음 보는 데이터이므로, 테스트 데이터셋의 정보(예: 테스트 문서 내 단어의 희귀성)를 미리 사용하여 모델을 조정하는 것은 허용되지 않습니다. 또한, 테스트 데이터셋이 단일 문서일 경우 IDF를 계산하는 것이 불가능하거나 의미가 없습니다.

4.0.3 5.3. TF-IDF 계산 및 예시

TF-IDF는 TF와 IDF를 곱하여 최종 단어 중요도 점수를 산출합니다.

TF-IDF 계산

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

- **예시 시나리오:** 4개의 문서로 구성된 코퍼스
 - 문서 1: "cat cat dog" (총 3단어)
 - 문서 2: "cat dog" (총 2단어)
 - 문서 3: "dog mouse" (총 2단어)
 - 문서 4: "dog" (총 1단어)
- **TF 계산(각 문서별):**
 - **문서 1:**
 - * $TF('cat', D1) = 2/3$
 - * $TF('dog', D1) = 1/3$
 - * $TF('mouse', D1) = 0/3 = 0$

- 문서 3:
 - * $TF('cat', D3) = 0/2 = 0$
 - * $TF('dog', D3) = 1/2$
 - * $TF('mouse', D3) = 1/2$
- **IDF 계산 (코퍼스 전체):** (총 문서 수 $N = 4$)
 - 단어 'cat': 문서 1, 2에 출현 ($DF('cat') = 2$)
 - * $IDF('cat') = \log(4/2) = \log(2) \approx 0.69$
 - * 해석: 'cat'은 4개 문서 중 2개에만 나타나므로, 문서 구별에 유용한 단어입니다.
 - 단어 'dog': 문서 1, 2, 3, 4에 출현 ($DF('dog') = 4$)
 - * $IDF('dog') = \log(4/4) = \log(1) = 0$
 - * 해석: 'dog'은 모든 문서에 나타나므로, 문서 구별에 전혀 도움이 되지 않는 단어입니다. TF-IDF 값은 0이 됩니다.
 - 단어 'mouse': 문서 3에 출현 ($DF('mouse') = 1$)
 - * $IDF('mouse') = \log(4/1) = \log(4) \approx 1.39$
 - * 해석: 'mouse'는 4개 문서 중 1개에만 나타나므로, 매우 희귀하며 문서 구별에 매우 유용한 단어입니다.
- **TF-IDF 계산 (문서 1의 'cat'):**
 - $TF-IDF('cat', D1) = TF('cat', D1) \times IDF('cat') = (2/3) \times 0.69 \approx 0.46$
 - 해석: 'cat'은 문서 1에서 자주 나타나고(TF 높음), 코퍼스 전체에서는 적당히 희귀하므로(IDF 높음), 문서 1을 대표하는 중요한 단어가 됩니다.

4.0.4 5.4. TF-IDF 변형 및 정규화

TF-IDF는 다양한 변형과 정규화 기법이 존재하며, 이는 실제 적용 시 성능에 영향을 미칠 수 있습니다.

TF-IDF 변형 및 정규화의 목적

TF-IDF 값의 계산 방식을 미세 조정하거나, 계산된 TF-IDF 벡터를 특정 스케일로 맞추어 모델의 안정성과 성능을 향상시키는 것이 목적입니다.

- **TF (Term Frequency) 변형:**
 - **Raw Count** (단순 빈도): $TF(t, d) =$ 단어 t 의 문서 d 내 출현 횟수 (Bag-of-Words와 유사)
 - **Boolean** (이진 빈도): $TF(t, d) = 1$ (단어 t 가 문서 d 에 출현하면), 0 (아니면)
 - **Logarithmic** (로그 빈도): $TF(t, d) = 1 + \log(\text{단어 } t \text{의 출현 횟수})$ (빈도가 높을수록 증가율 둔화)
 - **Double Normalization** (이중 정규화): $TF(t, d) = 0.5 + 0.5 \times \frac{\text{단어 } t \text{의 출현 횟수}}{\text{문서 } d \text{ 내 최대 단어 출현 횟수}}$ (과도한 빈도 편향 완화)
- **IDF (Inverse Document Frequency) 변형:**
 - **Smooth IDF** (스무스 IDF): $IDF(t) = \log\left(\frac{N+1}{DF(t)+1}\right) + 1$
 - * 분모가 0이 되는 것을 방지하고, IDF 값이 0이 되는 것을 막아 모든 단어에 최소한의 가중치를 부여합니다.
 - * scikit-learn의 TfidfVectorizer에서 기본값으로 사용됩니다.
 - **Probabilistic IDF** (확률적 IDF): 확률 이론에 기반한 변형으로, 단어가 나타나지 않는 문서의 비율을 고려합니다.

– **Maximal IDF:** 특정 단어의 IDF 값을 코퍼스 내 최대 IDF 값으로 정규화하는 방식입니다.

• **벡터 정규화 (Vector Normalization):**

– **목적:** 계산된 TF-IDF 벡터의 길이를 1로 만들어 단위 벡터(unit vector)로 변환합니다.

– **방법:** 각 벡터의 모든 요소를 해당 벡터의 L2 노름(Euclidean distance, 유클리드 거리)으로 나눕니다.

$$\text{정규화된 벡터 } \mathbf{v}' = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

– **L2 노름:** $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_k^2}$

– **효과:**

* **문서 길이의 영향 감소:** 문서의 길이가 길다고 해서 TF-IDF 벡터의 길이가 무조건 길어지는 것을 방지합니다.

* **방향성 강조:** 벡터의 '방향'에 초점을 맞춰 문서 간의 유사도를 측정할 때 '코사인 유사성(Cosine Similarity)'을 직접적으로 사용할 수 있게 합니다. 두 단위 벡터의 내적(dot product)은 두 벡터 사이의 코사인 값과 동일합니다.

* **모델 안정성:** 신경망의 입력으로 사용될 때, 입력 값의 스케일을 일정하게 유지하여 학습의 안정성을 높일 수 있습니다.

– **논의:** TF 계산 시 이미 문서 길이로 나누어 정규화하는데, 최종적으로 다시 벡터 정규화를 하는 것이 중복이 아닌가 하는 의문이 있을 수 있습니다. 하지만 TF는 '단어 빈도'를 문서 길이로 정규화하는 것이고, 최종 벡터 정규화는 '문서 전체의 TF-IDF 벡터'의 길이를 정규화하는 것으로, 서로 다른 목적을 가집니다. 경험적으로 두 가지 정규화를 모두 적용하는 것이 좋은 성능을 보이는 경우가 많습니다.

• **활용 분야:**

– **키워드 추출 (Keyword Extraction):** TF-IDF 값이 높은 단어는 문서의 핵심 키워드로 간주될 수 있습니다.

– **문서 분류 (Document Classification):** TF-IDF 벡터를 신경망이나 머신러닝 모델의 입력으로 사용하여 문서의 카테고리를 분류합니다.

– **검색 엔진 (Search Engine):** 사용자의 쿼리와 문서 간의 유사도를 TF-IDF 기반으로 계산하여 관련성 높은 문서를 검색 결과로 제공합니다.

5 6. 단어 임베딩 (Word Embedding)

단어 임베딩은 텍스트 데이터를 신경망이 이해할 수 있는 형태로 변환하는 핵심 기술입니다. 단어의 의미론적 관계를 저차원 밀집 벡터로 표현하여 텍스트 처리 모델의 성능을 혁신적으로 향상시켰습니다.

단어 임베딩의 개념

단어 임베딩 (Word Embedding)은 단어를 고차원의 희소한 원-핫 벡터가 아닌, 저차원의 밀집 (dense) 벡터로 표현하는 기법입니다. 이 과정에서 단어의 의미론적, 문맥적 관계가 벡터 공간에 반영되어, 유사한 의미를 가진 단어들은 벡터 공간에서 가까운 위치에 배치됩니다.

5.0.1 6.1. 원-핫 인코딩 (One-Hot Encoding)의 한계

단어 임베딩이 등장하기 전에는 주로 원-핫 인코딩을 사용했습니다.

- **표현 방식:** 어휘 (vocabulary)의 크기만큼의 차원을 가진 벡터에서, 해당 단어의 인덱스에만 1을 표시하고 나머지는 0으로 채웁니다.
 - 예: 어휘 = {'cat', 'dog', 'mouse'}
 - 'cat' $\rightarrow$ [1, 0, 0]
 - 'dog' $\rightarrow$ [0, 1, 0]
 - 'mouse' $\rightarrow$ [0, 0, 1]
- **문제점:**
 - **희소성 (Sparsity):** 어휘 크기가 커질수록 벡터의 대부분이 0이 되어 메모리 비효율적입니다.
 - **의미 관계 미반영:** 'cat'과 'dog'은 동물이라는 점에서 유사하지만, 원-핫 벡터 상에서는 서로 직교 (orthogonal)하여 아무런 의미론적 관계를 나타내지 못합니다. 모든 단어 간의 거리가 동일합니다.
 - **차원 문제:** 어휘 크기가 수만 개가 되면 입력 벡터의 차원이 너무 커져 신경망 학습에 부담이 됩니다.

5.0.2 6.2. 임베딩 레이어 (Embedding Layer)

단어 임베딩은 신경망의 한 부분으로 학습될 수 있습니다.

- **개념:** 원-핫 인코딩된 단어 벡터를 입력으로 받아, 이를 저차원의 밀집 벡터(임베딩 벡터)로 변환하는 신경망의 첫 번째 층입니다.
- **동작 원리:**
 - 입력: 어휘 크기 (V)의 원-핫 벡터 (예: [0, ..., 1, ..., 0])
 - 출력: 임베딩 차원 (D)의 밀집 벡터 (예: [0.1, -0.5, 0.3, ...])
 - 내부적으로 $V \times D$ 크기의 **임베딩 행렬 (Embedding Matrix) W_E** 를 가집니다. 원-핫 벡터가 입력되면, 해당 단어의 인덱스에 해당하는 임베딩 행렬의 행(row)을 찾아 출력합니다.
 - 이는 사실상 선형 변환 ($\mathbf{u} = \mathbf{x}W_E$)과 동일하며, 활성화 함수나 바이어스(bias)는 필요하지 않습니다.
- **학습:** 임베딩 행렬 W_E 의 값들은 신경망이 특정 작업(예: 분류, 예측)을 수행하는 과정에서 역전파 (backpropagation)를 통해 함께 학습됩니다. 즉, 임베딩은 '문제 해결' 과정의 일부로 최적화됩니다.
- **효율적인 구현:** 실제로는 고차원 원-핫 벡터를 곱하는 대신, 단어의 인덱스(예: 'cat'이 5번째 단어라면 인덱스 4)를 입력으로 받아 임베딩 행렬에서 해당 행을 직접 조회하는 방식으로 효율적으로 구현됩니다.

5.0.3 6.3. 임베딩 차원 결정

임베딩 벡터의 차원(D)은 중요한 하이퍼파라미터입니다.

- **결정 방법:** 정해진 규칙은 없으며, 주로 **경험적(empirical)**으로 결정됩니다.
- **일반적인 범위:** 어휘 크기에 따라 다르지만, 일반적으로 100차원, 200차원, 300차원 등 수백 차원 수준으로 설정됩니다.
- **고려 사항:**
 - 너무 낮으면 단어의 의미를 충분히 표현하지 못할 수 있습니다.
 - 너무 높으면 모델의 복잡도가 증가하고 과적합 위험이 있습니다.

5.0.4 6.4. 임베딩의 의미론적 특성

잘 학습된 단어 임베딩은 단어 간의 의미론적 관계를 벡터 공간에 반영합니다.

- **벡터 연산의 의미:** 임베딩 벡터 간의 덧셈, 뺄셈 연산이 의미 있는 결과를 도출할 수 있습니다.
- **예시:** "King - Man + Woman = Queen"
 - $\text{vec}(\text{King}) - \text{vec}(\text{Man})$ 은 '남성성'이라는 개념의 벡터를 나타냅니다.
 - 여기에 $\text{vec}(\text{Woman})$ 을 더하면, '여성성'을 가진 '왕'의 개념, 즉 '여왕(Queen)'에 해당하는 벡터를 얻을 수 있습니다.
 - 이는 벡터 공간에서 '남성-여성'이라는 관계가 일관된 '이동 벡터(shift vector)'로 표현된다는 것을 의미합니다.
- **유사성 측정:** 두 임베딩 벡터 간의 **코사인 유사성(Cosine Similarity)**을 통해 단어 간의 의미적 유사도를 측정합니다. 코사인 값이 1에 가까울수록 유사하고, 0에 가까울수록 관련성이 적습니다.

5.0.5 6.5. 단어 임베딩의 활용

단어 임베딩은 다양한 텍스트 처리 작업에 활용됩니다.

- **텍스트 표현:** 단어의 순서나 시간 정보를 보존하면서 텍스트를 효율적으로 표현할 수 있습니다. (예: 번역 모델)
- **문서 표현 (평균 임베딩):** 문서 내 모든 단어 임베딩 벡터의 평균을 계산하여 문서 전체를 하나의 벡터로 표현할 수 있습니다.
 - **장점:** 문서의 시간/순서 정보를 잃지만, Bag-of-Words나 TF-IDF보다 의미론적 정보를 더 잘 보존합니다.
 - **활용:** 문서 분류와 같이 순서 정보가 덜 중요하거나, 순환 신경망(RNN)과 같은 복잡한 모델 학습 비용이 부담될 때 사용될 수 있습니다.
- **다양한 NLP 태스크:** 감성 분석, 기계 번역, 질의응답 시스템 등 거의 모든 자연어 처리(NLP) 분야에서 핵심적인 입력 특징으로 사용됩니다.

5.0.6 6.6. 단어 임베딩의 한계

단어 임베딩은 강력하지만 몇 가지 한계점도 존재합니다.

- **OOV (Out-Of-Vocabulary) 문제:**
 - 훈련 데이터셋에 없는 새로운 단어(OOV)가 나타나면 해당 단어에 대한 임베딩 벡터를 생성할 수

없습니다.

- 해결책: OOV 단어를 위한 특별한 토큰(<UNK>)을 사용하거나, 서브워드(subword) 임베딩(예: Fast-Text)을 사용합니다.

- **정적 임베딩 (Static Embedding):**

- 전통적인 Word2Vec이나 GloVe 임베딩은 한 단어에 대해 하나의 고정된 벡터를 할당합니다.
- 문제점: 'bank'와 같이 여러 의미를 가지는 다의어(polysemy)의 경우, 문맥에 따라 다른 의미를 가지더라도 동일한 벡터로 표현됩니다.
- 해결책: 문맥을 고려하여 동적으로 임베딩을 생성하는 문맥 기반 임베딩(Contextualized Embedding, 예: ELMo, BERT)이 등장했습니다.

- **편향 반영 (Bias Reflection):**

- 임베딩은 훈련 데이터셋의 텍스트에서 단어의 사용 패턴을 학습합니다. 만약 훈련 텍스트에 사회적 편향(예: '의사'는 남성과, '간호사'는 여성과 자주 연결)이 있다면, 임베딩 벡터에도 이러한 편향이 반영됩니다.
- 문제점: 이는 모델이 편향된 예측이나 번역을 수행하게 만들 수 있습니다.
- 해결책: 편향을 제거하기 위한 다양한 후처리 기법이 연구되고 있습니다.

- **데이터 부족 언어:**

- 임베딩 모델은 대규모 텍스트 코퍼스에서 학습되어야 좋은 성능을 냅니다.
- 전자 형식의 텍스트 데이터가 부족한 언어의 경우, 고품질의 임베딩을 학습하기 어렵습니다.

5.0.7 6.7. 인기 있는 임베딩 모델

Word2Vec과 GloVe는 가장 널리 사용되는 단어 임베딩 모델입니다.

- **Word2Vec (Google):**

- 개념: 단어의 '주변 문맥(context)'을 기반으로 단어 임베딩을 학습합니다.
- 두 가지 학습 방식:
 1. **CBOW (Continuous Bag-of-Words):** 주변 단어들을 입력으로 받아 중심 단어를 예측합니다.
 2. **Skip-gram:** 중심 단어를 입력으로 받아 주변 단어들을 예측합니다. (일반적으로 더 좋은 성능을 보임)
- 학습 과정: 슬라이딩 윈도우(sliding window)를 텍스트에 적용하여, 각 윈도우 내의 단어들을 입력/출력 쌍으로 사용하여 신경망을 훈련합니다. 이 신경망의 은닉층 가중치가 임베딩 벡터가 됩니다.
- 특징: 단어 간의 의미론적 관계(예: King - Man + Woman = Queen)를 잘 포착합니다.

- **GloVe (Stanford):**

- 개념: 코퍼스 전체의 '동시 출현 통계(co-occurrence statistics)'를 활용하여 단어 임베딩을 학습합니다.
- 동작 원리:
 - * 코퍼스 내 모든 단어 쌍의 동시 출현 빈도를 계산하여 거대한 동시 출현 행렬(co-occurrence matrix)을 만듭니다.
 - * 이 행렬의 정보(로그 동시 출현 확률)를 임베딩 벡터의 내적(dot product)으로 모델링하도록 학습합니다.
- 특징: Word2Vec이 지역적인 문맥(sliding window)에 집중하는 반면, GloVe는 전역적인 코퍼스

통계를 활용합니다. 결과적으로 Word2Vec과 유사하게 의미론적 관계를 잘 포착합니다.

6 절차/방법: 텍스트 처리 실습

이 섹션에서는 scikit-learn의 TfidfVectorizer와 gensim의 Word2Vec을 사용하여 텍스트를 처리하고 임베딩을 생성하는 방법을 단계별로 설명합니다.

6.1 1. TF-IDF 벡터화 (scikit-learn TfidfVectorizer)

TfidfVectorizer는 텍스트 문서 집합을 TF-IDF 특징 행렬로 변환하는 데 사용됩니다.

□ TF-IDF 벡터화 기본 절차

1. TfidfVectorizer 객체 초기화 (필요에 따라 파라미터 설정).
2. fit() 메서드로 문서 집합에 대한 어휘(vocabulary)를 학습.
3. transform() 메서드로 문서를 TF-IDF 행렬로 변환.
4. get_feature_names_out()으로 학습된 어휘 확인.
5. toarray()로 희소 행렬을 밀집 행렬로 변환하여 결과 확인.

6.1.1 1.1. TfidfVectorizer 주요 파라미터

TfidfVectorizer를 초기화할 때 다양한 파라미터를 설정하여 텍스트 처리 방식을 제어할 수 있습니다.

- lowercase=True (기본값): 모든 텍스트를 소문자로 변환합니다.
- ngram_range=(1, 1) (기본값): 1-그램(단어 하나)만 사용합니다. (1, 2)로 설정하면 1-그램과 2-그램을 모두 사용합니다.
- stop_words=None (기본값): 불용어를 제거하지 않습니다. 'english'로 설정하면 영어 불용어를 제거합니다.
- token_pattern=r'(?u)\b\w\w+\b' (기본값): 토큰(단어)을 추출하는 정규 표현식입니다. 기본값은 두 글자 이상의 단어만 토큰으로 인식합니다. (예: 'a'와 같은 한 글자 단어는 제외)
- norm='l2' (기본값): TF-IDF 벡터를 L2 노름으로 정규화하여 단위 벡터로 만듭니다.

□ TF-IDF 벡터화 코드 예시

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # 문서집합코퍼스 ()
4 corpus = [
5     "a cat a dog and also my cats",
6     "a cat a dog and also my dogs",
7     "a dog a mouse and also my mice",
8     "a dog and also my dogs"
9 ]
10
11 # TfidfVectorizer 초기화기본값 ( 사용)
12 # lowercase=True, ngram_range=(1,1), stop_words=None, token_pattern=r'(?u)\
13     b\w\w+\b', norm='l2'
14 vectorizer = TfidfVectorizer()
```

```

14
15 # 문서집합에하고 fit 변환 (TF-IDF 행렬생성 )
16 tfidf_matrix = vectorizer.fit_transform(corpus)
17
18 # 학습된어휘단어 ( 목록 ) 확인
19 feature_names = vectorizer.get_feature_names_out()
20 print("어휘:", feature_names)
21
22 # TF-IDF 행렬출력밀집 ( 행렬로변환 )
23 print("TF-IDF 행렬:\n", tfidf_matrix.toarray())

```

Listing 1: TF-IDF 벡터화 기본 예시

코드 실행 결과 분석

- **어휘 (feature_names):** 'a'와 같은 한글자 단어는 기본 token_pattern에 의해 제외되고, 'also', 'and', 'cat', 'cats', 'dog', 'dogs', 'mice', 'mouse', 'my'와 같은 두 글자 이상의 단어들이 알파벳 순서로 어휘에 포함됩니다.
- **TF-IDF 행렬:** 각 행은 하나의 문서를, 각 열은 어휘의 단어를 나타냅니다. 셀 값은 해당 문서에서 단어의 TF-IDF 점수입니다.
- **정규화 확인:** norm='l2'가 기본값이므로, 각 문서 벡터의 L2 노름(길이)은 1이 됩니다. (예: 첫 번째 문서 벡터의 각 요소를 제공하여 더한 후 제곱근을 취하면 1이 나옴)

6.1.2 1.2. 불용어 (Stop Words) 제거

문서 구별에 도움이 되지 않는 흔한 단어(불용어)를 제거하여 차원을 줄이고 모델 성능을 향상시킬 수 있습니다.

□ 불용어 제거 코드 예시

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 corpus = [
4     "a cat a dog and also my cats",
5     "a cat a dog and also my dogs",
6     "a dog a mouse and also my mice",
7     "a dog and also my dogs"
8 ]
9
10 # TfidfVectorizer 초기화영어 ( 불용어제거 )
11 vectorizer_sw = TfidfVectorizer(stop_words='english')
12
13 tfidf_matrix_sw = vectorizer_sw.fit_transform(corpus)
14 feature_names_sw = vectorizer_sw.get_feature_names_out()
15 print("불용어 제거후어휘 :", feature_names_sw)
16 print("불용어 제거후 TF-IDF 행렬:\n", tfidf_matrix_sw.toarray())
17

```

```

18 # sklearn-의 learn 영어불용어목록확인
19 from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
20 print("\nsklearn 영어불용어목록일
    부      ():\n", list(ENGLISH_STOP_WORDS)[:10])

```

Listing 2: 영어 불용어 제거 예시

코드 실행 결과 분석

- 'a', 'and', 'also', 'my'와 같은 단어들이 불용어 목록에 포함되어 제거됩니다.
- 어휘의 크기가 'cat', 'cats', 'dog', 'dogs', 'mice', 'mouse' 등으로 훨씬 짧아집니다.
- 차원이 줄어들어 계산 효율성이 높아집니다.
- 사용자 정의 불용어: stop_words 파라미터에 리스트 형태로 직접 불용어 목록을 전달하여 사용자 정의 불용어를 제거할 수도 있습니다. (예: stop_words=['cat', 'dog'])

6.1.3 1.3. N-그램 적용

단어의 순서 정보를 일부 보존하기 위해 N-그램을 사용할 수 있습니다.

□ 2-그램 적용 코드 예시

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 corpus = [
4     "a cat a dog and also my cats",
5     "a cat a dog and also my dogs",
6     "a dog a mouse and also my mice",
7     "a dog and also my dogs"
8 ]
9
10 # TfidfVectorizer 초기화(2-그램만 사용)
11 vectorizer_ngram = TfidfVectorizer(ngram_range=(2, 2))
12
13 tfidf_matrix_ngram = vectorizer_ngram.fit_transform(corpus)
14 feature_names_ngram = vectorizer_ngram.get_feature_names_out()
15 print("그램2- 어휘:", feature_names_ngram)
16 print("그램2- TF-IDF 행렬:\n", tfidf_matrix_ngram.toarray())

```

Listing 3: 2-그램 TF-IDF 벡터화 예시

코드 실행 결과 분석

- 어휘가 'also my', 'and also', 'cat a', 'cat and', 'dog a', 'dog and', 'my cats', 'my dogs', 'my mice'와 같은 2-그램으로 구성됩니다.
- 어휘의 크기가 1-그램을 사용할 때보다 훨씬 커질 수 있습니다.
- 'not bad'와 같이 부정적인 의미를 가지는 구문을 하나의 토큰으로 처리하여 문맥을 더 잘 반영할

수 있습니다.

- `ngram_range=(1, 2)`로 설정하면 1-그램과 2-그램을 모두 어휘에 포함시킬 수 있습니다.

6.2 2. NLTK를 이용한 텍스트 전처리 (스테밍)

NLTK (Natural Language Toolkit)는 텍스트 전처리에 유용한 도구들을 제공합니다. 여기서는 스테밍 (Stemming)을 예시로 들어봅니다.

□ NLTK 스테밍 코드 예시

```

1  import nltk
2  from nltk.stem import PorterStemmer
3  from nltk.tokenize import word_tokenize
4  from sklearn.feature_extraction.text import TfidfVectorizer
5
6  # NLTK 데이터다운로드최초 ( 회1 실행)
7  # nltk.download('punkt')
8
9  corpus = [
10     "a cat a dog and also my cats",
11     "a cat a dog and also my dogs",
12     "a dog a mouse and also my mice",
13     "a dog and also my dogs"
14 ]
15
16 # Porter Stemmer 초기화
17 stemmer = PorterStemmer()
18
19 # 문서전처리함수토큰화 ( -> 스테밍 -> 재결합)
20 def preprocess_document(doc):
21     tokens = word_tokenize(doc) # 단어토큰화
22     stemmed_tokens = [stemmer.stem(token) for token in tokens] # 각토큰에스
        테밍적용
23     return " ".join(stemmed_tokens) # 스테밍된토큰들을다시문장으로결합
24
25 # 코퍼스내모든문서에전처리적용
26 preprocessed_corpus = [preprocess_document(doc) for doc in corpus]
27 print("전처리된 코퍼스:", preprocessed_corpus)
28
29 # 전처리된문서에 TfidfVectorizer 적용
30 vectorizer_stem = TfidfVectorizer(token_pattern=r'(?u)\b\w+\b') # 한글자단어
        도포함하도록패턴변경
31 tfidf_matrix_stem = vectorizer_stem.fit_transform(preprocessed_corpus)
32 feature_names_stem = vectorizer_stem.get_feature_names_out()
33 print("스테밍 후어휘 :", feature_names_stem)
34 print("스테밍 후 TF-IDF 행렬:\n", tfidf_matrix_stem.toarray())

```

Listing 4: NLTK를 이용한 스테밍 및 TF-IDF 적용

코드 실행 결과 분석

- **스테밍 효과:** 'cats' → 'cat', 'dogs' → 'dog', 'mice' → 'mic' (Porter Stemmer의 결과)과 같이 단어의 접미사가 제거되어 어간(stem)만 남습니다.
- **어휘 감소:** 'cat'과 'cats'가 'cat'으로 통합되면서 어휘의 크기가 줄어들고, 동일한 의미를 가진 단어들이 하나의 토큰으로 처리됩니다.
- `tokenpattern` : `token_pattern` , 'a' `r'(?u)\b\w+\b'` .
- **표제어 추출 (Lemmatization):** 스테밍보다 더 정교하게 단어의 기본형(표제어)을 찾아주는 기법입니다. (예: 'better' → 'good'). NLTK의 WordNetLemmatizer를 사용할 수 있습니다.

7 3. Word2Vec 임베딩 (gensim Word2Vec)

gensim 라이브러리는 Word2Vec 모델을 쉽게 구현할 수 있도록 돕습니다.

□ Word2Vec 임베딩 기본 절차

1. 텍스트 데이터를 문장 단위로 토큰화하고, 각 문장을 단어 리스트로 만듭니다.
2. `gensim.models.Word2Vec` 객체 초기화 (주요 파라미터: `vector_size`, `window`, `min_count`).
3. `build_vocab()` 메서드로 어휘를 구축.
4. `train()` 메서드로 모델을 학습.
5. 학습된 모델에서 단어 임베딩 벡터 추출 및 유사 단어/유추 문제 해결.

7.0.1 3.1. Word2Vec 주요 파라미터

- `vector_size`: 임베딩 벡터의 차원 (예: 2, 100, 200). 실습에서는 시각화를 위해 2차원을 사용하지만, 실제로는 100 300차원을 권장합니다.
- `window`: 중심 단어 주변에서 고려할 단어의 최대 거리. (예: `window=5`는 중심 단어 좌우 5개 단어까지 고려)
- `min_count`: 모델이 학습할 단어의 최소 빈도. 이 값보다 적게 나타나는 단어는 무시됩니다.
- `sg`: 학습 알고리즘 선택. 0은 CBOW, 1은 Skip-gram (기본값).
- `epochs`: 훈련 반복 횟수.

□ Word2Vec 임베딩 코드 예시

```

1  import nltk
2  from nltk.tokenize import word_tokenize
3  from gensim.models import Word2Vec
4  import numpy as np
5
6  # NLTK 데이터다운로드최초 ( 회1 실행)
7  # nltk.download('punkt')
8
9  # 예시문서허리케인 ( 관련뉴스 )
10 documents = [
11     "Florida is bracing for Hurricane Ian, which is expected to make
12     landfall as a major storm.",
13     "Residents are urged to evacuate coastal areas as the storm approaches.",
14     "",
15     "Power outages are widespread, affecting thousands of homes.",
16     "The storm surge could cause significant flooding.",
17     "Emergency services are on high alert, preparing for the worst."
18 ]
19
20 # 문서전처리토큰화 ( 및소문자변환 )
21 tokenized_sentences = [word_tokenize(doc.lower()) for doc in documents]
22 print("토큰화된 문장:", tokenized_sentences)

```

```

21
22 # Word2Vec 모델 학습
23 # vector_size=2: 시각화를 위해 차원 2 임베딩 사용 실제로는 (더 높은 차원 사용)
24 # window=5: 중심 단어 좌우 개 5 단어까지 문맥으로 고려
25 # min_count=1: 최소 빈 1 이상 나타나는 단어만 학습
26 model = Word2Vec(sentences=tokenized_sentences, vector_size=2, window=5,
27                  min_count=1, sg=1)
28
29 # 모델 학습
30 model.train(tokenized_sentences, total_examples=model.corpus_count, epochs=
31             model.epochs)
32
33 # 학습된 어휘 확인
34 vocabulary = list(model.wv.index_to_key)
35 print("\n학습된 어휘 일부 ():", vocabulary[:10])
36
37 # 특정 단어의 임베딩 벡터 추출
38 word_vectors = {word: model.wv[word] for word in ['storm', 'power', '
39             residents', 'outages']}
40 print("\n'storm' 임베딩 벡터 :", word_vectors['storm'])
41 print("'power' 임베딩 벡터 :", word_vectors['power'])
42 print("'residents' 임베딩 벡터 :", word_vectors['residents'])
43 print("'outages' 임베딩 벡터 :", word_vectors['outages'])

```

Listing 5: Word2Vec 모델 학습 및 임베딩 추출

코드 실행 결과 분석

- 토큰화된 문장: 각 문장이 단어 리스트로 분리되고 소문자로 변환됩니다.
- 학습된 어휘: 모델이 학습한 단어들이 어휘에 포함됩니다.
- 임베딩 벡터: 'storm', 'power', 'residents', 'outages'와 같은 단어들이 2차원 벡터로 표현됩니다. 이 벡터 값들은 모델 학습 과정에서 단어의 문맥을 반영하여 결정됩니다.
- 시각화: 2차원 임베딩은 X-Y 평면에 단어를 점으로 찍어 시각화할 수 있습니다. 유사한 단어는 가까이 위치하고, 관련 없는 단어는 멀리 떨어져 있을 것입니다.

7.0.2 3.2. 단어 유사성 계산

Word2Vec 모델은 단어 간의 코사인 유사성을 기반으로 가장 유사한 단어를 찾아주는 기능을 제공합니다.

□ 단어 유사성 계산 코드 예시

```

1 # 'destruction'과 '가장 유사한 단어 찾기
2 # 예시( 텍스트에 'destruction'이 없으므로, 강의에서 언급된 'distraction'으로 대체)
3 # 또는 'storm'과 유사한 단어를 찾아봅니다
4 similar_words_storm = model.wv.most_similar('storm', topn=2)
5 print("\n'storm'과 '가장 유사한 단어 상위 (개2):", similar_words_storm)

```

```

6
7 # 결과예시 : [('waters', 0.99...), ('frustration', 0.98...)]
8 # 이는 'storm과 'waters가 ' 벡터공간에서가장가까운코사인 ( 유사성이높은 ) 단어
   임을의미합니다 .

```

Listing 6: 가장 유사한 단어 찾기

7.0.3 3.3. 단어 유추 (Analogy) 해결

Word2Vec 모델은 벡터 연산을 통해 단어 유추 문제(예: King - Man + Woman = ?)를 해결할 수 있습니다.

단어 유추 문제 해결 원리

단어 유추 문제는 "A는 B에 해당하는 것처럼, C는 D에 해당한다"는 관계를 찾는 것입니다. 이를 벡터 연산으로 표현하면 $\text{vec}(A) - \text{vec}(B) \approx \text{vec}(C) - \text{vec}(D)$ 가 됩니다. 따라서 D 를 찾으려면 $\text{vec}(D) \approx \text{vec}(C) - \text{vec}(A) + \text{vec}(B)$ 를 계산한 후, 이 결과 벡터와 가장 가까운 단어를 찾습니다.

□ 단어 유추 해결 코드 예시

```

1 # 예시텍스트에 'king', 'man', 'woman', 'queen이' 없으므로,
2 # 강의에서언급된 'power', 'outage', 'fuel을' 사용하여유추문제를만들어봅니다 .
3 # "는power 에outage 해당하는것처럼 , 은fuel 무엇에해당하는가 ?"
4 # 즉,  $\text{vec}(\text{outage}) - \text{vec}(\text{power}) + \text{vec}(\text{fuel})$  에가장가까운단어를찾습니다 .
5
6 #  $\text{model.wv.most\_similar}(\text{positive}=[\text{vec}(C), \text{vec}(B)], \text{negative}=[\text{vec}(A)], \text{topn}=1)$ 
7 #  $(C + B - A)$ 
8 analogy_result = model.wv.most_similar(positive=['outage', 'fuel'],
    negative=['power'], topn=1)
9 print("\n'power가' 'outage에' 해당하는것처럼 , 'fuel은' 무엇에해당하는
    가 ?:", analogy_result)
10
11 # 결과예시 : [('remained', 0.99...)]
12 # 이는텍스트의문맥상 'power outage와' 'fuel remained가' 유사한관계를가질수있음
    을시사합니다 .
13 # 매우 ( 작은코퍼사이므로결과가직관적이지않을수있습니다 .)

```

Listing 7: 단어 유추 문제 해결 (King - Man + Woman

8 실습/코드/오류와 해결

이 섹션에서는 강의 중 발생한 코드 관련 논의와 실제 구현 시 발생할 수 있는 일반적인 오류 및 해결책을 다룹니다.

8.1 1. TF-IDF 벡터 정규화 논의

강의 중 TF-IDF 벡터의 정규화에 대한 논의가 있었습니다.

TF-IDF 벡터 정규화 불일치 논의

강의 슬라이드에 제시된 TF-IDF 값들이 L2 노름으로 정규화되지 않은 것처럼 보였으나, `scikit-learn`의 `TfidfVectorizer`는 `norm='l2'`를 기본값으로 사용하여 각 문서 벡터를 L2 노름으로 정규화합니다. 학생의 코드 실행 결과, 벡터의 L2 노름이 1에 가까운 값(예: $\sqrt{0.8944^2 + 0.4472^2} \approx 1$)이 나왔음을 확인했습니다. 이는 슬라이드의 숫자가 정규화 전의 값이거나, 강사의 타이핑 오류였음을 의미합니다.

- **증상:** 슬라이드에 제시된 TF-IDF 벡터 값들의 제곱합의 제곱근이 1이 아니었습니다.
- **원인 가설:**
 1. `TfidfVectorizer`의 `norm` 파라미터가 `None`으로 설정되었을 가능성.
 2. 슬라이드에 제시된 숫자가 정규화되기 전의 값일 가능성.
 3. 강사의 타이핑 오류.
- **수정 방법 및 결과 검증:**
 1. `TfidfVectorizer`의 공식 문서를 확인한 결과, `norm='l2'`가 기본값임을 확인했습니다.
 2. 학생이 직접 코드를 실행하여 확인한 결과, L2 노름으로 정규화된 단위 벡터가 출력됨을 확인했습니다.
- **결론:** `TfidfVectorizer`는 기본적으로 L2 노름 정규화를 수행하여 각 문서 벡터의 길이를 1로 만듭니다. 이는 코사인 유사성 계산 시 벡터의 방향성만을 고려하게 하여 문서 간 유사도 측정에 유리합니다.

8.2 2. ChatGPT 언어별 성능 논의

강의 말미에 ChatGPT의 언어별 성능에 대한 흥미로운 논의가 있었습니다.

□ ChatGPT 언어별 성능 및 특징

ChatGPT와 같은 대규모 언어 모델(LLM)은 훈련 데이터의 양과 질에 따라 언어별로 성능 차이를 보입니다. 영어는 방대한 양의 고품질 텍스트 데이터로 훈련되어 매우 자연스럽게 정확한 응답을 생성하는 반면, 다른 언어들은 상대적으로 데이터가 부족하여 미묘한 차이가 발생할 수 있습니다.

- **영어:** 매우 사실적이고 사람과 대화하는 듯한 자연스러움을 보입니다. 문법적 오류가 거의 없으며, 다양한 문맥을 이해하고 적절히 반응합니다.
- **아랍어:** 특정 방언까지 구사할 수 있을 정도로 뛰어난 성능을 보입니다. 다만, 남성/여성 구분이 중요한 일부 단어에서 미묘한 오류가 발생할 수 있습니다.
- **힌디어:** 매우 세련되고 정제된 언어를 구사하여, 마치 '교수님'이 말하는 듯한 느낌을 줍니다. 이는 훈련 데이터가 주로 책이나 공식 문서와 같은 정제된 텍스트였기 때문으로 추정됩니다. 일상적인

대화체와는 다를 수 있습니다.

- **일반적인 문제점:**

- **데이터 부족:** 영어 외의 언어는 훈련 데이터의 양이 적어 성능이 떨어질 수 있습니다.
- **편향 반영:** 훈련 데이터에 존재하는 편향(예: 특정 성별에 대한 고정관념)이 모델의 응답에 반영될 수 있습니다.
- **최신 정보 부족:** 특정 시점까지의 데이터로 훈련되므로, 그 이후의 최신 정보는 알지 못합니다.

- **활용 팁:**

- **프롬프트 개선 요청:** ChatGPT에게 ”프롬프트를 어떻게 개선할 수 있을까요?”라고 물어보면 더 나은 질문을 만드는 데 도움을 받을 수 있습니다.
- **오류 지적:** ”어떤 실수를 했나요?” 또는 ”어떤 부정확한 정보를 제공했나요?”라고 물으면 스스로 오류를 지적하고 수정하는 모습을 보입니다.
- **역할 부여:** 특정 역할(예: ”전문가처럼 설명해 줘”)을 부여하면 그에 맞는 어조와 깊이로 답변을 생성합니다.

9 체크리스트

□ 학습 및 과제 점검 체크리스트

이 체크리스트를 활용하여 강의 내용을 효과적으로 복습하고 과제를 완벽하게 준비하세요.

- 개념 이해
 - 가중치 공유의 개념과 RNN, CNN에서의 적용 방식 이해
 - N-그램 토큰화의 필요성 및 오버랩핑 N-그램의 의미 이해
 - TF-IDF의 TF, IDF 각각의 정의와 계산 방법 이해
 - TF-IDF 계산 시 훈련/테스트 데이터셋 활용 방식 이해 (IDF는 훈련 데이터 기반)
 - 단어 임베딩의 필요성 (원-핫 인코딩의 한계) 이해
 - 임베딩 레이어의 동작 원리 및 임베딩 벡터의 의미론적 특성 이해
 - Word2Vec (CBOW, Skip-gram) 및 GloVe의 기본 원리 이해
- 실습 및 코드
 - TfidfVectorizer를 이용한 TF-IDF 행렬 생성 코드 구현 및 결과 해석
 - TfidfVectorizer의 ngram_range, stop_words, token_pattern 파라미터 활용
 - NLTK를 이용한 텍스트 토큰화 및 스테밍/표제어 추출 전처리 코드 구현
 - gensim.models.Word2Vec을 이용한 단어 임베딩 모델 학습 코드 구현
 - 학습된 Word2Vec 모델에서 단어 임베딩 벡터 추출 및 시각화 (개념적)
 - Word2Vec 모델의 most_similar() 메서드를 이용한 유사 단어 찾기 및 단어 유추 문제 해결
- 과제 준비
 - 다음 과제 마감일 (다다음 주 일요일) 확인 및 일정 계획
 - 과제에서 요구하는 TF-IDF 및 임베딩 적용 방식 확인 (예: 10,000개 단어 사용)
 - 90% 정확도 달성 목표 확인 (지난 과제보다 쉬울 수 있음)
- 심화 학습
 - TF-IDF의 다양한 변형(로그 TF, 스무스 IDF 등) 및 벡터 정규화의 효과 탐색
 - 단어 임베딩의 한계(OOV, 정적 임베딩, 편향) 및 해결 방안 추가 조사
 - Word2Vec과 GloVe의 내부 알고리즘 상세 비교

10 FAQ (자주 묻는 질문)

□ FAQ: 초심자를 위한 질문과 답변

초심자들이 텍스트 처리 및 임베딩을 학습하면서 흔히 가질 수 있는 질문들을 모아 답변합니다.

- Q: TF-IDF에서 IDF를 계산할 때 왜 테스트 데이터셋이 아닌 훈련 데이터셋을 사용하나요?** **A:** IDF는 단어 자체의 희귀성(전체 코퍼스 내에서의 분포)을 측정하는 값입니다. 이는 모델이 학습하는 '지식'의 일부로 간주됩니다. 테스트 데이터셋은 모델이 '처음 보는' 데이터여야 하므로, 테스트 데이터셋의 정보를 미리 사용하여 IDF를 계산하는 것은 모델의 일반화 성능을 왜곡할 수 있습니다. 또한, 테스트 데이터셋이 단일 문서일 경우 IDF를 계산하는 것이 불가능합니다. 따라서 IDF는 훈련 데이터셋에서만 한번만 계산하여 고정적으로 사용합니다.
- Q: TF-IDF 벡터를 L2 노름으로 정규화하는 것이 왜 중요한가요?** **A:** L2 노름 정규화는 TF-IDF 벡터의 길이를 1로 만들어 단위 벡터로 변환합니다. 이는 문서의 길이가 길다고 해서 TF-IDF 벡터의 길이가 무조건 길어지는 것을 방지하여, 문서 길이의 영향을 줄입니다. 또한, 벡터의 '방향'에 초점을 맞추게 되어, 문서 간의 유사도를 코사인 유사성으로 측정할 때 벡터의 크기가 아닌 순수한 방향성만을 비교할 수 있게 합니다. 이는 모델의 안정성과 성능 향상에 기여합니다.
- Q: Word2Vec과 같은 단어 임베딩이 Bag-of-Words나 TF-IDF보다 왜 더 좋다고 평가되나요?** **A:** Bag-of-Words와 TF-IDF는 단어의 출현 빈도나 희귀성만을 고려하며, 단어 간의 의미론적 관계(예: '왕'과 '여왕'의 관계)를 전혀 포착하지 못합니다. 반면, 단어 임베딩은 단어를 저차원의 밀집 벡터로 표현하면서 단어의 문맥 정보를 학습하여, 유사한 의미를 가진 단어들이 벡터 공간에서 가깝게 위치하도록 만듭니다. 이는 모델이 텍스트의 의미를 더 깊이 이해하고, 번역, 감성 분석 등 다양한 NLP 태스크에서 훨씬 뛰어난 성능을 발휘하게 합니다.
- Q: 임베딩 레이어를 신경망에 직접 추가하여 학습하는 것과 Word2Vec처럼 별도로 사전 학습된 임베딩을 사용하는 것의 차이는 무엇인가요?** **A:** 신경망에 임베딩 레이어를 직접 추가하여 학습하는 방식은 '작업 특정(task-specific)' 임베딩을 생성합니다. 즉, 현재 해결하려는 특정 문제(예: 영화 리뷰 분류)에 최적화된 임베딩을 학습합니다. 이 방식은 문맥 정보를 명시적으로 고려하지 않고, 오직 최종 비용 함수를 최소화하는 방향으로 임베딩을 조정합니다. 반면, Word2Vec과 같은 사전 학습된 임베딩은 대규모 코퍼스에서 '단어의 문맥'을 예측하는 비지도 학습 방식으로 학습됩니다. 이는 단어 간의 일반적인 의미론적 관계를 포착하며, 다양한 작업에 전이 학습(transfer learning)될 수 있습니다. Word2Vec은 단어 간의 상관관계를 명시적으로 고려하므로, 동의어 등이 유사한 벡터를 가질 가능성이 더 높습니다.
- Q: 단어 임베딩의 '편향 반영' 문제는 어떻게 발생하며 왜 중요한가요?** **A:** 단어 임베딩은 훈련 데이터셋의 텍스트에서 단어의 사용 패턴을 학습합니다. 만약 훈련 텍스트에 사회적 편향(예: '의사'라는 단어가 남성 대명사와 더 자주 함께 사용됨)이 존재한다면, 임베딩 모델은 이러한 편향을 그대로 학습하여 '의사'와 '남성'의 임베딩 벡터를 가깝게 만들 수 있습니다. 이는 모델이 편향된 예측(예: 직업 추천 시 성별 편향)을 하거나, 번역 시 성별을 잘못 유추하는 등의 문제를 야기할 수 있습니다. 따라서 이러한 편향을 인식하고 제거하려는 노력이 중요합니다.

11 빠르게 훑어보기 (1페이지 요약)

□ 핵심 개념 요약: 텍스트 처리 및 임베딩

CSCI89 Lecture 5

1. 가중치 공유 (Weight Sharing)

- 개념: 신경망의 여러 연결에서 동일 가중치 사용. 파라미터 수 감소, 패턴 재사용.
- 적용:
 - FNN: 없음. 각 연결 고유 가중치.
 - RNN: 있음. 시퀀스 데이터에서 시간 독립적 패턴 학습.
 - CNN: 있음. 이미지에서 위치 독립적 특징 추출 (필터 슬라이딩).

2. N-그램 토큰화 (N-gram Tokenization)

- 개념: N개의 연속 단어 묶음. 단어 순서 정보 일부 보존.
- 예시 (2-그램): "Port assembly line" → "Port assembly", "assembly line".
- 개수: 길이 L 에서 N-그램은 $L - N + 1$ 개. 실제 어휘는 1-그램보다 N-그램이 훨씬 많음.

3. TF-IDF (Term Frequency-Inverse Document Frequency)

- 목표: 단어의 중요도 측정. Bag-of-Words의 한계 극복.
- TF (Term Frequency): 문서 내 단어 빈도 / 문서 총 단어 수.
- IDF (Inverse Document Frequency): $\log(\text{전체 문서 수} / \text{단어 포함 문서 수})$. 희귀할수록 높음.
- 계산: $TF \times IDF$.
- 특징: IDF는 훈련 데이터셋 기반으로 한 번만 계산.
- 정규화: L2 노름으로 벡터 길이 1로 정규화 (단위 벡터). 코사인 유사성 측정에 유리.

4. 단어 임베딩 (Word Embedding)

- 목표: 단어를 저차원 밀집 벡터로 표현, 의미 관계 포착.
- 원-핫 인코딩 한계: 희소성, 의미 관계 미반영.
- 임베딩 레이어: 신경망의 한 층으로 학습. 가중치 행렬이 임베딩 행렬.
- 의미론적 특성: 벡터 연산으로 단어 관계 표현 (예: King - Man + Woman = Queen).
- 한계: OOV, 정적 임베딩, 편향 반영, 데이터 부족 언어.
- 주요 모델:
 - Word2Vec: 주변 단어로 중심 단어 예측 (CBOW) 또는 중심 단어로 주변 단어 예측 (Skip-gram).
 - GloVe: 코퍼스 전체의 동시 출현 통계 기반.

5. 실습 도구

- **TF-IDF:** `sklearn.feature_extraction.text.TfidfVectorizer`
- **전처리:** NLTK (토큰화, 스테밍)
- **Word2Vec:** `gensim.models.Word2Vec`